

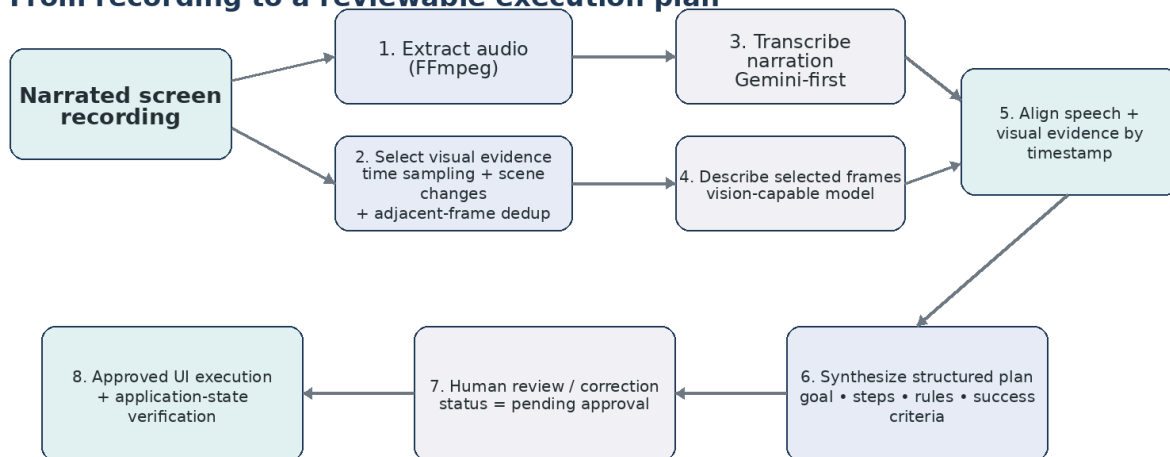
Demonstration-Guided Agentic Automation

Turning Screen Recordings into Executable Business Processes
When Application APIs or MCP Tools Are Not Available

Show the process. Let the agent execute it.

No application-specific API. No MCP business-action tool. Live UI execution under the user context.

From recording to a reviewable execution plan



Staged interpretation keeps the original recording as evidence while separating perception, planning, approval and execution.

September 2026 | POC / Architecture Paper

Abstract

Most enterprise automation assumes that the desired process has already been translated into a specification: an API contract, an MCP tool, a workflow definition, or an RPA script. **This paper explores a different starting point: the business user simply demonstrates the work.** A narrated screen recording becomes the source evidence from which an AI system extracts intent, observes application state, generates a reviewable execution plan, and—after approval—executes the business process through the live application user interface.

The POC is intentionally focused on a gap scenario: **business actions for which an application-specific API or MCP tool is unavailable or not practical.** Model APIs are still used for interpretation and reasoning; the distinction is that the business action itself is carried out through the UI. This turns demonstration into a candidate specification while preserving review, user-context identity, verification, and explicit failure handling.

Core thesis: Demonstration can become a first-class input to automation—but only when perception, plan generation, approval, execution and verification are treated as separate architectural concerns.

1. The automation gap

Enterprise systems contain a long tail of repetitive work that is easy for a person to perform but expensive to formalize. The problem is rarely that the steps are individually difficult. The problem is that someone must discover the right records, interpret context, navigate the application, apply judgment, and repeat the same sequence over and over.

The conventional choices each solve part of the problem:

- API-first automation is reliable and scalable when the required business capability is exposed cleanly.
- MCP can make application capabilities easier for agents to discover and invoke when those capabilities are already available as tools.
- Traditional RPA can automate UI sequences, but it often depends on brittle selectors, fixed paths, and workflow logic authored in advance.
- A human can always operate the UI, but the repetitive effort and operational variance remain.

The POC investigates a complementary pattern: **use the business user's narrated demonstration to infer a process, convert it into an explicit plan, and let an agent work toward that intent against the live application state.**

2. Demonstration becomes evidence—not magic

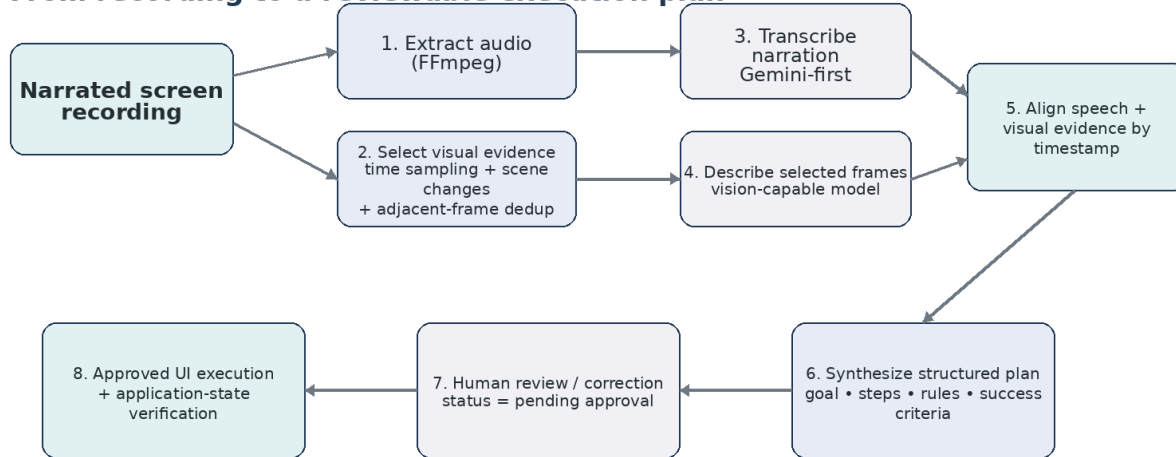
A screen recording contains two different forms of information that become much more valuable when interpreted together:

- Narration carries business intent: why the user chose a record, what qualifies as urgent, which values are examples, and when an exception applies.
- Visual evidence carries application state and demonstrated action: what was on screen, which object was selected, what changed, and what result was visible.

The architecture does not treat a recording as a command to blindly replay clicks. Instead, it treats the recording as evidence from which a structured, reviewable process model can be synthesized. That distinction is essential: one observed example does not automatically become a universal business rule.

3. From recording to understanding

From recording to a reviewable execution plan



Staged interpretation keeps the original recording as evidence while separating perception, planning, approval and execution.

Figure 1. Staged interpretation pipeline used by the POC.

3.1 Extract the narration

The audio track is extracted from the uploaded recording with FFmpeg. The implementation then attempts Gemini-based transcription first, with local transcription fallbacks available. The transcript is retained with timing information so spoken intent can later be associated with the visual evidence occurring at roughly the same point in the process.

3.2 Extract the right frames—not every frame

Frame extraction is one of the most important parts of the design. Instead of sending the entire video as one opaque request, the pipeline selects evidence in stages:

1. Time sampling provides baseline coverage across the recording.
2. Scene-change detection adds frames when the visual state materially changes.
3. Adjacent-frame deduplication removes near-identical screenshots so the model is not repeatedly shown the same application state.
4. The remaining frames are sent to a vision-capable model for concise descriptions of what is visible and what action appears to have occurred.

This produces a compact visual timeline rather than a raw stream of screenshots. It also makes the interpretation process easier to inspect: important UI state can be traced back to selected evidence frames and timestamps.

Why staged processing? It reduces redundant visual evidence, preserves timestamped traceability, allows audio and vision stages to be inspected independently, and gives review/correction a concrete evidence trail.

3.3 Align what the user said with what the user did

For each selected frame, the system associates nearby narration segments using timestamps. This multimodal alignment matters because neither channel is sufficient by itself. A screen may show a user opening a Case, while the narration explains that the case is being selected because it threatens a high-value deal. Conversely, narration may say “create a follow-up task,” while the visual evidence reveals the specific fields and application context involved.

The goal is not video summarization. The goal is process reconstruction: business intent + visual state + demonstrated action + evidence timestamp.

4. The Plan is the execution contract

The output of interpretation is not an immediate browser action. It is a structured Plan that separates understanding from execution. In the current POC, the Plan contains:

- Business goal and summary.
- Typed execution steps.
- Inferred decision rules and optional explanations.
- Required credential providers.
- Per-step success conditions and failure policies.
- Source recording reference, version, status, and correction history.

Newly generated plans begin in a pending-approval state. The user can review the inferred process and submit corrections in natural language. Corrections regenerate and version the plan before it can be approved for execution. This is a critical guardrail: the system makes uncertainty visible rather than silently converting ambiguous demonstrations into business policy.

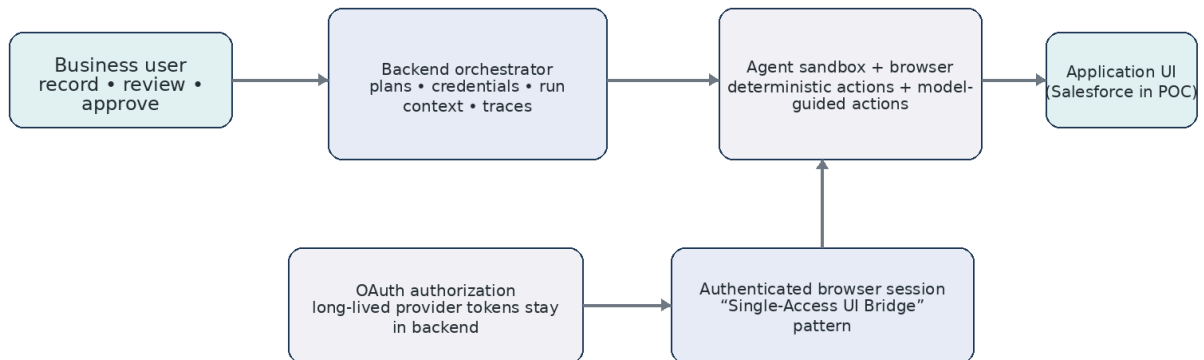
Illustrative plan contract

Field	Purpose
goal	Outcome the automation is trying to achieve.
steps[]	Ordered or conditional actions with intent and action type.
rules[]	Inferred criteria, exceptions, variables, or unresolved assumptions.
success_condition	Observable application-state evidence of completion.
failure_policy	Pause, retry, skip, or abort behavior.

Field	Purpose
source_evidence	Recording, captions, timestamps, and plan version used to derive the step.
status	Pending approval, approved, executed, corrected, etc.

5. Agent execution through the live UI

Execution path: approved intent, live UI state, user-context identity



Business actions are performed through the live application UI. Model APIs still support interpretation/reasoning; the “no API/MCP” claim applies only to the business-action execution path.

Figure 2. Approved plan execution under a user-context authenticated browser session.

The run coordinator starts a browser sandbox and supplies the approved plan plus run context. The executor supports two complementary execution modes:

- Deterministic sequences for known interactions that can be executed predictably without a model call for every sub-action.
- Model-guided browser actions for situations where the agent must observe the current screen, choose the next action, execute it, and observe again.

The important architectural point is that the agent works toward the intent of the approved step against the current interface. It does not simply replay the original recording’s cursor coordinates. Navigation, waits, extraction, branching, loops, and explicit human-input pauses can be represented as execution behavior.

5.1 What “No API / No MCP” means in this paper

The claim is deliberately narrow. The selected POC does not require an application-specific API or MCP tool to perform the business action itself—for example, editing a Case or creating a Task. Those actions are executed through the application UI. However, model APIs still support transcription, vision interpretation, planning, and model-guided reasoning, while backend APIs/services handle authentication, orchestration, persistence, and execution control.

This is not an argument against APIs or MCP. When reliable business-action tools exist, they are often the preferred execution path. UI-native agent execution is a complementary pattern for the long tail where those tools are missing, incomplete, delayed, or impractical.

6. Identity, authorization and the Single-Access UI Bridge

The POC executes inside the business user's Salesforce context. The user authorizes a Salesforce connection through OAuth. Long-lived provider tokens remain protected in the backend. When an automation run begins, the backend establishes an authenticated browser session using Salesforce's single-access login mechanism and directs the sandbox browser into the resulting application session.

For the purpose of this architecture, this is described as a "Single-Access UI Bridge": a bridge from the user's authorized identity to a browser session in which the agent performs the approved workflow. The term is descriptive, not an official Salesforce product name.

Identity alone is not sufficient authorization. The platform still has to enforce which plan/version the user approved, which actions are allowed, what the browser session can access, and how run tokens, live-view access, and network exposure are protected. Production hardening remains required.

7. POC walkthrough: a Salesforce case process

The demo uses a deliberately ordinary business process because the value is easiest to see in repetitive work. A business user demonstrates a daily Salesforce workflow in which they:

1. Review Cases and identify an urgent Acme purchase-order issue.
2. Recognize that the issue affects a \$285,000 deal and needs attention.
3. Update/escalate the Case and associate the relevant customer context.
4. Create and assign a follow-up Task.

The automation then executes the equivalent business outcome through the UI. The demo is intentionally framed as "show the work once, then let the agent handle the repetition." The technical significance is not the specific Case workflow; it is that the demonstration becomes evidence for a reusable execution plan rather than a one-time recording or a hard-coded click script.

8. Where this pattern fits—and where it does not

Pattern	Best fit	Strength	Primary trade-off
API / MCP-first	Stable, exposed business capabilities	Deterministic contracts; scalable execution	Requires the needed capability to already be exposed
Traditional RPA	Highly repetitive, predictable UI paths	Mature orchestration for scripted processes	Can be brittle when UI/state varies; logic must be authored
Demonstration-guided agentic UI execution	Long-tail processes where business action tools are unavailable or incomplete	Starts from business-user evidence; can reason over live state	Needs stronger verification, recovery, security, and bounded autonomy

The approach is especially interesting for legacy systems, internal applications, niche SaaS workflows, and "last-mile" tasks that remain manual because engineering a dedicated integration cannot be justified. It is

less appropriate for irreversible or high-risk actions unless the architecture adds stronger approval, policy, simulation, transaction, and rollback controls.

9. Verification matters more than the agent's explanation

A browser action succeeding is not the same as the business outcome succeeding. The POC therefore treats application state as the source of truth for completion. Examples include verifying that:

- The intended record actually changed.
- Records that should have been excluded remained unchanged.
- A task or record was not duplicated.
- Missing information or ambiguity was surfaced instead of invented.
- The observed end state satisfies the step's declared success condition.

Execution traces, model reasoning text, timing, and tool observations are useful for debugging and explanation, but they are not independent proof that the business action was correct. This distinction becomes increasingly important as the level of autonomy increases.

10. Memory and learning from prior runs

The backend can retrieve procedural and episodic hints from earlier executions before a new run and store lessons after execution. This is retrieval-based memory, not model retraining. Its value should be measured empirically: does prior-run context reduce retries, improve recovery, or increase business-outcome accuracy without encouraging stale assumptions?

11. Current POC boundary and production roadmap

The implementation is an architecture POC, not a production automation platform. The reviewed snapshot uses a FastAPI backend, Next.js UI, SQL-backed persistence, local recording artifacts, local Docker browser sandboxes, and in-process background execution. Runs are initiated on demand; durable queues, production-scale scheduling/event triggers, and full infrastructure isolation remain future work.

The next stage should focus less on “can the agent click the UI?” and more on enterprise operational guarantees:

- Evidence-backed rule extraction: explicitly store record-selection rules, variable inputs, exclusions, evidence timestamps, and unresolved questions.
- Immutable approval binding: execute exactly the plan version the user approved.
- Reliability: resumable state, nested-step recovery, idempotency, duplicate prevention, and deterministic success checks.
- Security: hardened sandbox isolation, least-privilege sessions, run-token protection, endpoint controls, auditability, and live-view governance.
- Policy and transaction guardrails: pre-execution checks, simulation where feasible, blast-radius limits, and rollback/compensation patterns for higher-risk actions.
- Measurement: task success, selection errors, retries, time-to-automation, execution latency, cost, human corrections, and drift across UI changes.

12. Architectural principles

Separate interpretation from execution. The business user should be able to inspect and correct what the system inferred before records are changed.

Use a structured plan as the contract. Perception and browser execution can evolve independently when they share an explicit intermediate representation.

Treat demonstrations as examples, not universal rules. The system must distinguish what happened once from what should happen every time.

Execute against live state, not recorded coordinates. The agent should pursue step intent based on the current interface and application state.

Keep user identity and automation approval distinct. Being able to authenticate as the user does not mean every possible action is authorized.

Verify business outcomes in the application. Observed end state—not model confidence or narrative—is the completion criterion.

Prefer APIs/MCP when they are the better tool. UI execution should complement strong application capabilities, not replace them by default.

Conclusion

The most interesting shift is not “AI that clicks.” It is the possibility that a business user’s demonstration can become a governed input to automation. A narrated recording can supply intent and evidence; selected frames can reveal state transitions; a multimodal model can synthesize a candidate plan; a human can review and correct it; and an agent can then execute the approved process against the live UI and verify the result.

That opens a practical path for the long tail of enterprise work that sits between manual operation and fully engineered integration. The architecture still requires rigorous approval, identity, verification, security, and recovery—but it reduces one of the hardest parts of automation: forcing the business user to translate how work is actually done into an implementation specification before automation can even begin.

Show the process. Capture the intent. Review the plan. Let the agent execute. Verify the outcome.

Appendix A — Share-ready summary

Demonstration-Guided Agentic Automation explores a simple question: what if a business user could teach an agent by showing it how work gets done? In this POC, a narrated screen recording is decomposed into audio and visual evidence. The system transcribes the narration, samples and deduplicates meaningful frames, uses vision to interpret application state, aligns what the user said with what they did, and synthesizes a reviewable execution plan. Once approved, an agent executes the process through the live application UI under the user's authenticated context and verifies the actual end state.

The selected experiment intentionally performs the business actions without requiring an application-specific API or MCP tool. Model APIs still power interpretation and reasoning; the business operation itself happens through the UI. The goal is not to replace strong APIs or MCP capabilities, but to address the long tail of processes that remain manual because those capabilities do not yet exist or are not practical to build.

Suggested video / paper caption

Show the process. Let the agent execute it.

A business user demonstrates a repetitive Salesforce process once. The system extracts narration and key frames, reconstructs intent, generates a reviewable plan, and then an agent executes the approved workflow directly through the UI. No application-specific API or MCP business-action tool is required for the execution path.